

Diagonal collapse for iterated suprema

a visual companion to the Lean development

Abstract

This document draws the mathematics of the Lean files in `RequestProject/`: the theorem

$$(\forall i\, j, \exists k, F_{ij} \leq F_{kk}) \implies \bigsqcup_i \bigsqcup_j F_{ij} = \bigsqcup_k F_{kk},$$

its two structural ingredients (currying and cofinality), the conditionally complete variant and where its boundedness hypothesis is consumed, the dependency DAG of the twenty-five lemmas as an actual graph, the seven bridges to other contexts as arrow diagrams, and the catalogue of alternative proofs as a Boolean lattice of design choices. Every picture corresponds to a compiled Lean declaration; the caption names it.

Contents

1	The statement, drawn	3
1.1	Three readings of the same supremum	3
1.2	The lattice picture of <code>le_antisymm</code>	3
1.3	A concrete instance: $\mathbb{R}_{\geq 0}^\infty$ and the sum of two suprema	3
1.4	Where boundedness enters	4
1.5	Two concrete lattices	4
1.6	The analytic corollary: a monotone staircase	4
2	The proof space as a graph	7
2.1	Connectivity: a three-colouring, and the two bridges that kill it	7
2.2	Why connectivity is proved by descent, not by search	7
2.3	Cofinality as a bipartite graph	7
3	Arrow diagrams	12
3.1	The proofs as commuting triangles	12
3.2	The binary-operation form as a zig-zag	12
3.3	Currying and Fubini for colimits	12
3.4	The categorical bridge: the collapse <i>is</i> finality of Δ	12
3.5	Transport along the bridges	13
3.6	The bridge hub	13
4	The space of proofs of one lemma	15
4.1	A fourth axis: duality	15
4.2	From the abstraction to the applications	16
4.3	Inventory	16

Where things live.

<code>RequestProject/Diagonal.lean</code>	the core: currying, cofinality, boundedness
<code>RequestProject/Applications.lean</code>	$\mathbb{R}_{\geq 0}^{\infty}$, $\mathbb{N}_{\infty}$, Cardinal , one topological corollary
<code>RequestProject/Bridges.lean</code>	the same theorem in seven other contexts
<code>RequestProject/ProofGraph.lean</code>	the dependency DAG, with decide -checked invariants
<code>RequestProject/Variations.lean</code>	~40 alternative proofs
<code>RequestProject/MathlibPR.lean</code>	the upstreaming staging file

1 The statement, drawn

Everything in `RequestProject/Diagonal.lean` orbits one picture: a doubly indexed family $F : \iota \rightarrow \iota \rightarrow \alpha$ is a *square* of lattice elements, and the hypothesis

$$h : \forall i j, \exists k, F_{ij} \leq F_{kk}$$

says that the *diagonal* of that square is cofinal in it. The conclusion $\bigsqcup_i \bigsqcup_j F_{ij} = \bigsqcup_k F_{kk}$ says the square and its diagonal have the same supremum.

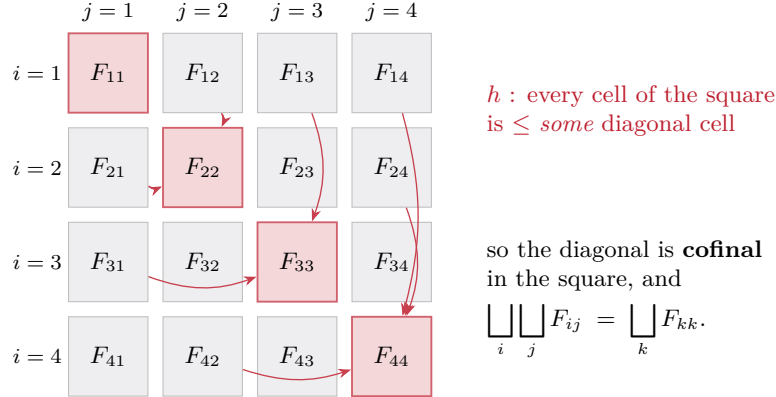


Figure 1: The diagonal hypothesis of `Golf.iSup2_eq_iSup_diagonal` as a combinatorial statement about a square: the red cells absorb everything.

1.1 Three readings of the same supremum

The proof of the collapse factors through a change of index: first the square is read as one family over the product index (`Golf.iSup_pprod`, *currying*), then the diagonal is shown cofinal in it (`Golf.iSup_eq_iSup_of_cofinal`).

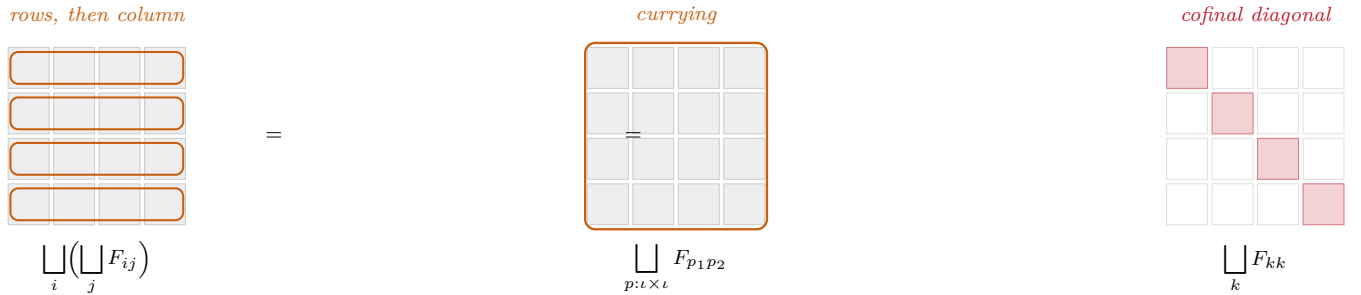


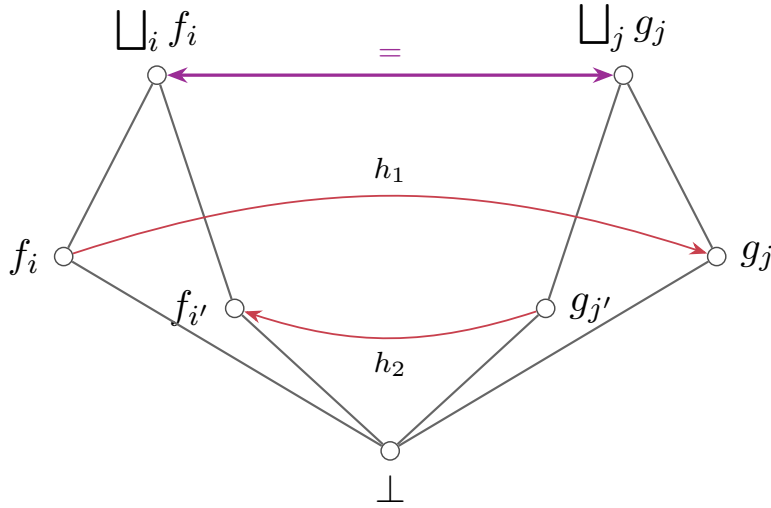
Figure 2: The two structural steps of the development. The first equality is `iSup_pprod` (true always); the second is `iSup_eq_iSup_of_cofinal` (true under h).

1.2 The lattice picture of `le_antisymm`

Both structural lemmas are proved by antisymmetry: two families with a supremum each, and two cofinality arrows. As a Hasse diagram, the proof squeezes two elements together.

1.3 A concrete instance: $\mathbb{R}_{\geq 0}^\infty$ and the sum of two suprema

The headline application $(\bigsqcup_i f_i) + (\bigsqcup_j g_j) = \bigsqcup_k (f_k + g_k)$ is the square of sums $F_{ij} = f_i + g_j$: monotone f, g over a directed index make the diagonal cofinal.



`iSup_eq_iSup_of_cofinal`
`≤ : iSup_mono' h1`
`≥ : iSup_mono' h2`
 two arrows, no witnesses

Figure 3: Mutual cofinality forces equality of suprema: each element of one family has an upper bound in the other, so neither supremum can be the larger.

	$g_1 = 0$	$g_2 = 2$	$g_3 = 5$	$g_4 = 7$	
$f_1 = 1$	1	3	6	8	
$f_2 = 3$	3	5	8	10	
$f_3 = 4$	4	6	9	11	
$f_4 = 6$	6	8	11	13	

f, g monotone $\Rightarrow$ the largest entry of the square is the last diagonal one:
 $\max_{i,j}(f_i + g_j) = f_4 + g_4 = 13$
`ennreal_iSup_add_iSup,`
`enat_iSup_add_iSup,`
`cardinal_ciSup_add_ciSup_diagonal`

Figure 4: A numerical square of sums. Monotonicity of the two families is exactly what makes the red diagonal absorb the square, which is why `ennreal_iSup_add_iSup_of_monotone` needs no diagonal hypothesis.

1.4 Where boundedness enters

In a *conditionally* complete lattice a supremum only behaves well below a ceiling. The development is arranged so that the ceiling is used exactly once, in `Golf.ciSup_pprod`; everything downstream inherits it.

1.5 Two concrete lattices

The Boolean lattice $2^{\{a,b,c\}}$ makes the collapse tangible: take the two chains $f = (\{a\}, \{a, b\})$ and $g = (\{c\}, \{b, c\})$. Every join $f_i \sqcup g_j$ lies below the last diagonal join $f_2 \sqcup g_2 = \{a, b, c\}$, so the double supremum collapses.

The hypothesis is not decoration. In the three-element chain $0 < 1 < 2$, put $F_{11} = F_{22} = 0$ and $F_{12} = F_{21} = 2$: no diagonal entry dominates F_{12} , and the conclusion fails.

1.6 The analytic corollary: a monotone staircase

For monotone $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}^{\infty}$ the diagonal family $k \mapsto f_k + g_k$ is monotone too, so it converges to its supremum — which, by the collapse, is $(\bigsqcup_i f_i) + (\bigsqcup_j g_j)$. That is `Golf.ennreal_tendsto_add_atTop`.

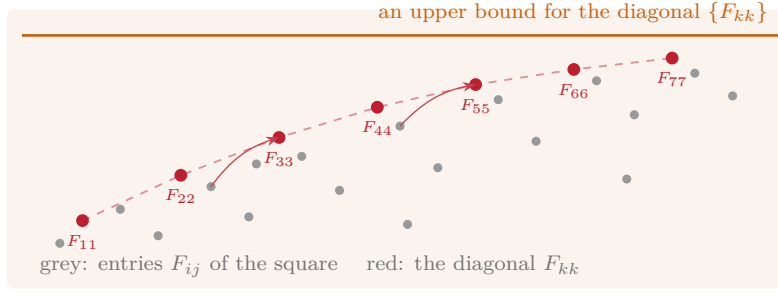


Figure 5: Boundedness travels *down* cofinality. `bddAbove_range_uncurry` turns the diagonal hypothesis into `isCofinalFor_range_iff` and then applies `IsCofinalFor.bddAbove`, transporting one ceiling from the diagonal to the whole square. That single transport is what feeds `ciSup_pprod`, `bddAbove_row` and `bddAbove_range_ciSup_row`: the user of `ciSup2_eq_ciSup_diagonal` only ever has to bound the diagonal.

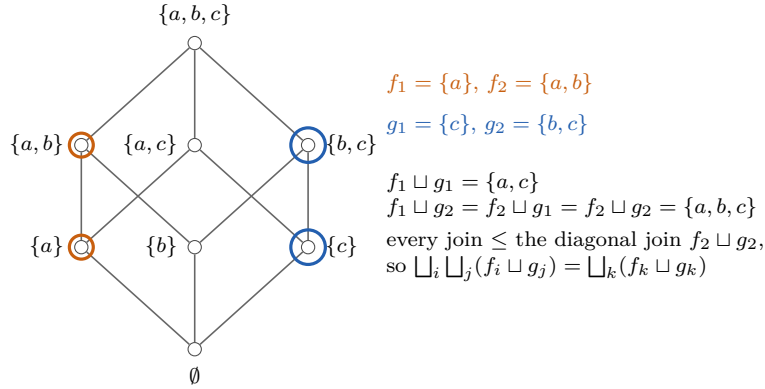


Figure 6: The diagonal hypothesis holds automatically here because both families are monotone and the index order is directed — the situation of `ennreal_iSup_add_iSup_of_monotone` and `cardinal_ciSup_add_ciSup_of_monotone`.

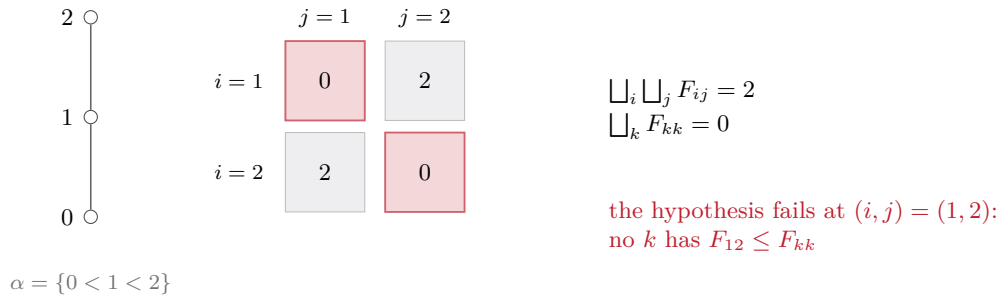


Figure 7: Why `Golf.iSup2_eq_iSup_diagonal` needs its hypothesis. Formally, the $\geq$ direction is free (`iSup_mono` with the witness $\langle k, k \rangle$); it is the $\leq$ direction that consumes h .

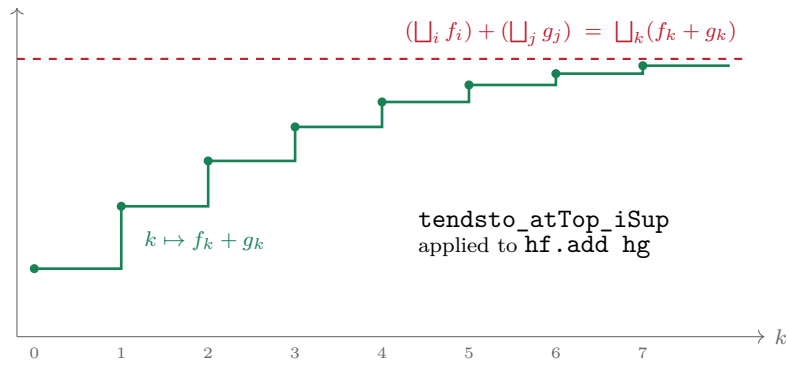


Figure 8: The collapse is what identifies the limit of the diagonal sequence with the sum of the two separate suprema; without it one only gets convergence to $\bigsqcup_k (f_k + g_k)$.

2 The proof space as a graph

`RequestProject/ProofGraph.lean` reifies the spine of the development as a `SimpleGraph` on 25 nodes and proves its structural properties by `decide`. This section draws that graph. Node colour is the tag, horizontal position is the rank (= height in the DAG, `Golf.Graph.rank`), and an arrow $b \rightarrow a$ means “ a uses b ” (`Golf.Graph.Uses`).

■ atomic	Mathlib, or one tactic	10 nodes	<code>card_atomic</code>
■ reducible	an iso/dual/rewrite collapses it	3 nodes	
■ glue	a small composition	7 nodes	
■ structural	introduces a new invariant	5 nodes	<code>card_structural</code>

2.1 Connectivity: a three-colouring, and the two bridges that kill it

The interesting finding of `ProofGraph.lean` is negative: the use-graph is *disconnected*. The categorical route and the linear-order route reach the same theorems through disjoint material. This is proved by a graph colouring invariant (`colour_eq_of_adj`, checked edge by edge with `decide`, then propagated along walks in `colour_eq_of_reachable`).

2.2 Why connectivity is proved by descent, not by search

Reachability is expensive for the kernel, so the positive statement is reduced to a local one: a parent function `toRoot` together with a strictly decreasing `depth`. This is a spanning tree of the bridged graph, rooted at `le_antisymm`.

2.3 Cofinality as a bipartite graph

The hypothesis pair $h_1 : \forall i, \exists j, f_i \leq g_j$ and $h_2 : \forall j, \exists i, g_j \leq f_i$ of `Golf.iSup_eq_iSup_of_cofinal` is a combinatorial condition on a bipartite graph: every vertex on each side has at least one out-neighbour on the other.

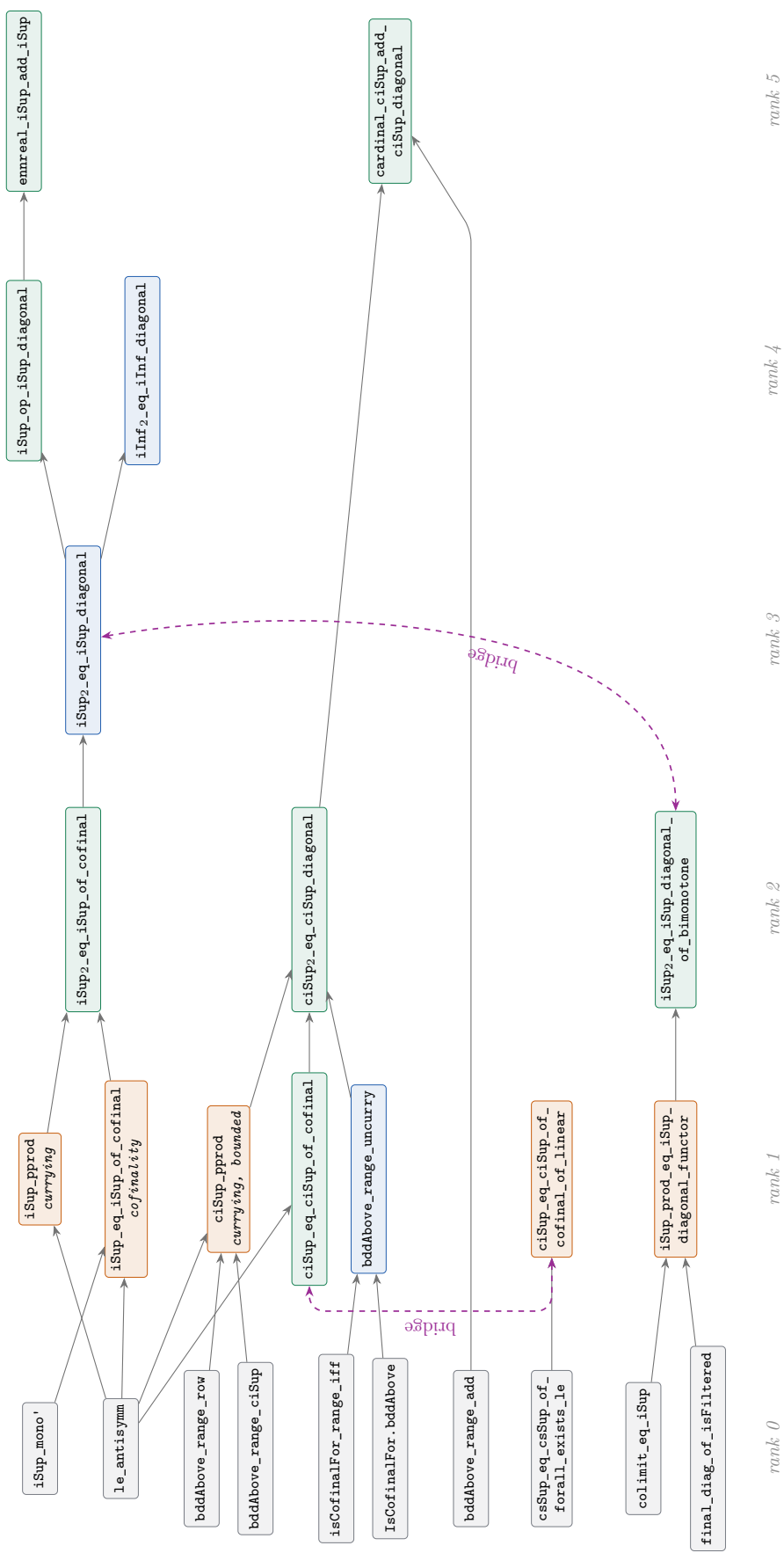


Figure 9: The dependency DAG of the development (Golf.Graph.G), with the two bridge links of Golf.Graph.G' dashed in purple. `dep_rank_lt` states that every solid arrow goes strictly left-to-right; `atomic_iff_leaf` that the grey nodes are exactly those with no incoming arrow; `deps_length_le_three` that no node has more than three incoming arrows; `card_edges` that there are 24 solid edges and `card_bridged_edges` that the bridges cost exactly two more.

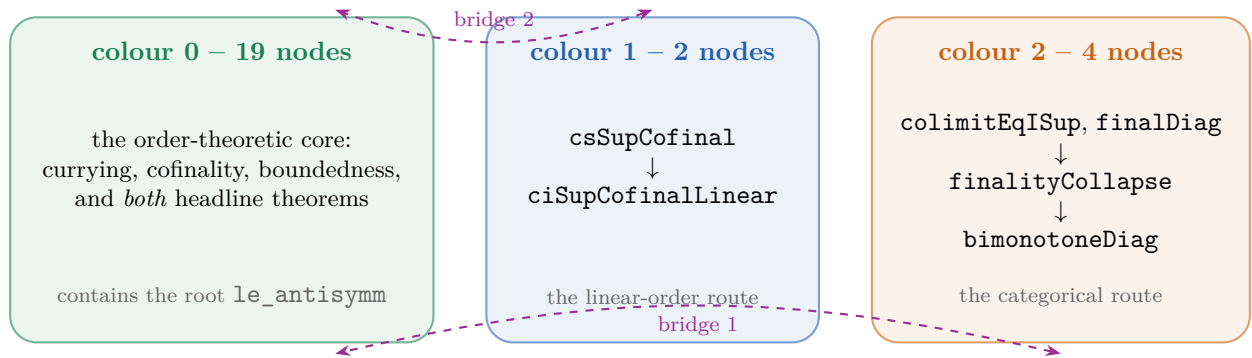


Figure 10: `not_connected`: the colouring is constant on connected components and takes three values, so $\mathcal{G}$ is disconnected — the categorical proof of the collapse shares *no* lemma with the order-theoretic one. Adding the two purple bridge edges makes the space connected (`bridged_connected`).

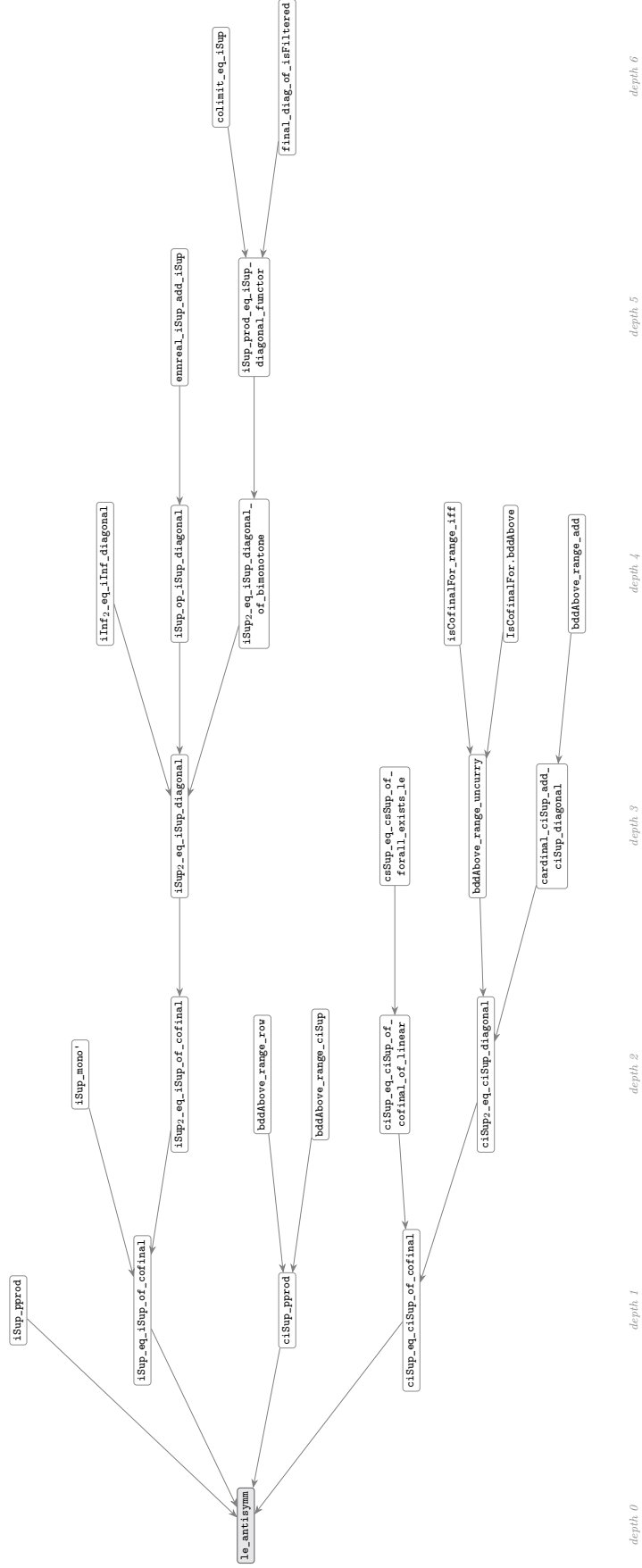


Figure 11: The spanning tree given by `toRoot`: every node other than the root is adjacent in G' to its parent (`adj_toRoot`) and strictly closer to the root (`depth_toRoot_lt`). Strong induction on `depth` then gives `reachable_root` and hence `bridged_connected`. Note that the tree uses *both* bridge edges, which is why it does not exist in G .

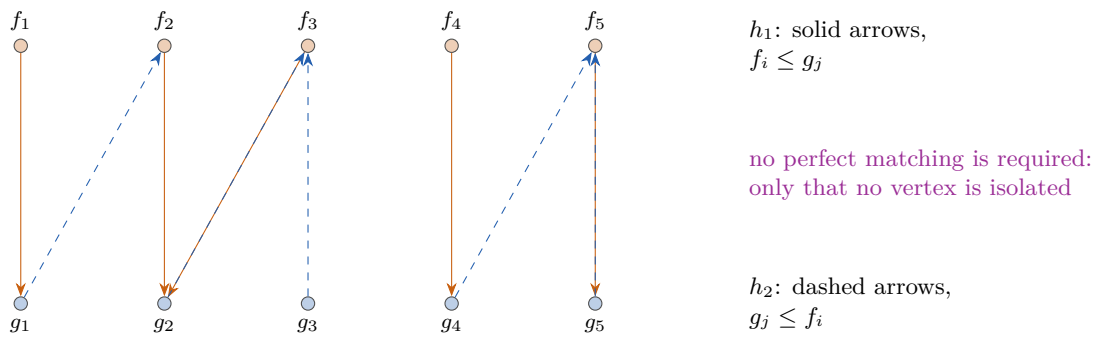


Figure 12: Mutual cofinality. The proof never chooses a canonical partner: it applies `iSup_mono'` twice, which consumes exactly the existence of *some* out-neighbour.

3 Arrow diagrams

A complete lattice is a category with all (co)limits: objects are elements, there is one arrow $a \rightarrow b$ exactly when $a \leq b$, $\sqcup$ is the colimit and $\sqcap$ the limit. Under that dictionary each lemma of the development becomes a diagram, and each bridge of `RequestProject/Bridges.lean` becomes a functor.

3.1 The proofs as commuting triangles

Both collapse lemmas are a `trans` of two equalities, i.e. a commuting triangle whose slanted edge is the theorem.

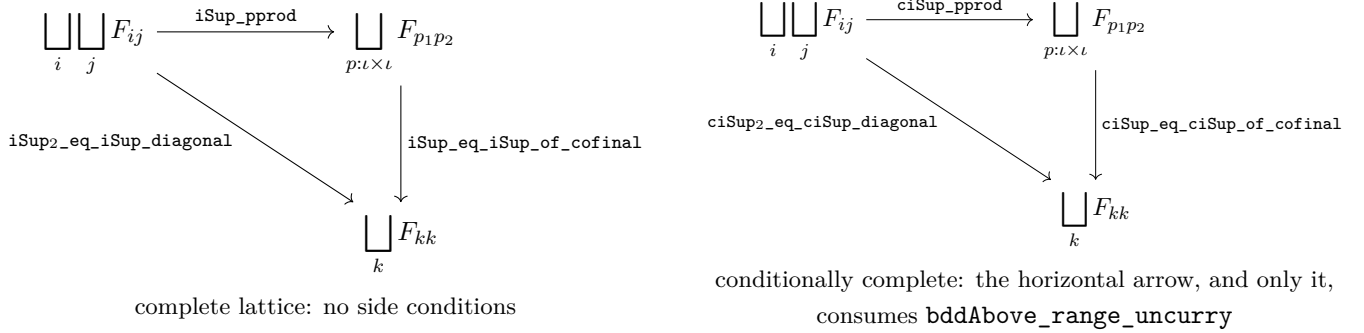


Figure 13: The same triangle twice. Passing from the complete to the conditionally complete world changes one edge label, not the shape.

3.2 The binary-operation form as a zig-zag

`Golf.iSup_op_iSup_diagonal` takes the two distributivity laws of the ambient theory as parameters and produces the diagonal form; the proof is a three-step chain.

$$\begin{array}{ccc}
 \text{op}(\bigsqcup_i f_i)(\bigsqcup_j g_j) & \xrightarrow{h_\ell} & \bigsqcup_i \text{op}(f_i)(\bigsqcup_j g_j) \\
 & & \downarrow \bigsqcup_i h_r \\
 \bigsqcup_k \text{op}(f_k)(g_k) & \xleftarrow{\text{iSup2_eq_iSup_diagonal}} & \bigsqcup_i \bigsqcup_j \text{op}(f_i)(g_j)
 \end{array}$$

Figure 14: Instantiating `op` with `(.+)` and the pair `ENNReal.iSup_add/ENNReal.add_iSup` gives `ennreal_iSup_add_iSup`; with `(.*.)` and `ENNReal.iSup_mul/ENNReal.mul_iSup` the multiplicative analogue — same proof, new inputs.

3.3 Currying and Fubini for colimits

`Golf.iSup_pprod` is the lattice shadow of the Fubini theorem for colimits: the colimit over a product category is the iterated colimit.

3.4 The categorical bridge: the collapse *is* finality of Δ

For a bimonotone family over a directed (= filtered) index, no order-theoretic argument is needed at all: Δ is a final functor, and finality is precisely invariance of colimits: this is `iSup_prod_eq_iSup_diagonal_functor` in `Bridges.lean`.

$$\begin{array}{ccc}
\iota \times \iota & \xrightarrow{F} & \alpha \\
\text{pr}_1 \downarrow & \searrow \sqcup_j F(-,j) & \\
\iota & &
\end{array}
\qquad
\begin{array}{ccc}
\iota & \xrightarrow{\Delta} & \iota \times \iota \\
& \searrow F \circ \Delta & \downarrow F \\
& & \alpha
\end{array}$$

Figure 15: Left: the left Kan extension along pr_1 is the row-supremum, and its colimit is $\sqcup_i \sqcup_j F_{ij}$. Right: the diagonal functor $\Delta = \text{CategoryTheory.Functor.diag}$, whose composite with F is the diagonal family. The theorem says the two colimits agree.

$$\begin{array}{ccccc}
\iota & \xrightarrow{\Delta} & \iota \times \iota & \xrightarrow{\text{colim}} & \text{colim } F \\
& \searrow F\Delta & \downarrow F & & \parallel \text{Functor.Final.colimitIso} \\
& & \alpha & \xrightarrow{\text{colim}} & \text{colim}(F\Delta) \\
\text{colim } F & = \bigsqcup_p F_p & & \text{colimit_eq_iSup} & \\
\text{colim } F\Delta & = \bigsqcup_k F_{kk} & & \text{colimit_eq_iSup} & \\
\Delta \text{ final} & \Leftarrow \iota \text{ filtered} & & \text{final_diag_of_isFiltered} &
\end{array}$$

Figure 16: The categorical route. Directedness of ι enters as filteredness of the index category; cofinality of the diagonal *set* becomes finality of the diagonal *functor*.

3.5 Transport along the bridges

Each bridge of `Bridges.lean` is a map of contexts. What differs between them is how much of the statement crosses: an order isomorphism carries hypothesis *and* conclusion, a sup-homomorphism only the conclusion, and duality carries everything for free because it is the identity on propositions.

$$\begin{array}{ccc}
\begin{array}{ccc}
\alpha & \xrightarrow[e]{\sim} & \beta \\
\sqcup \downarrow & & \downarrow \sqcup \\
\alpha & \xrightarrow[e]{} & \beta
\end{array} &
\begin{array}{ccc}
\alpha & \xrightarrow{e} & \beta \\
\sqcup \downarrow & & \downarrow \sqcup \\
\alpha & \xrightarrow[e]{} & \beta
\end{array} &
\begin{array}{ccc}
& \overset{l}{\curvearrowright} & \\
\alpha & \xrightarrow[\perp]{u} & \beta \\
\sqcup \downarrow & & \downarrow \sqcup \\
\alpha & \xrightarrow[l]{} & \beta
\end{array} \\
\text{OrderIso:} & \text{sSupHom: not injective;} & \text{GaloisConnection: the left} \\
\text{e.map_iSup, both ways} & \text{only the conclusion crosses} & \text{adjoint preserves } \sqcup
\end{array}$$

Figure 17: Three functorial bridges of increasing weakness. In all three the square commutes, which is all the proof of the transported collapse uses (`simp only` [`<- map_iSup`, `iSup2_eq_iSup_diagonal`]).

3.6 The bridge hub

$$\begin{array}{ccc}
\alpha & \xrightarrow[\sim]{(-)^{\text{od}}} & \alpha^{\text{od}} \\
\downarrow \sqcup_i \sqcup_j F_{ij} = \sqcup_k F_{kk} & & \downarrow \prod_i \prod_j F_{ij} = \prod_k F_{kk} \\
\text{Prop} & \xlongequal{\text{Iff.rfl}} & \text{Prop}
\end{array}$$

Figure 18: The duality bridge (`Golf.Bridge.iSup2_diagonal_iff_dual`). The $\prod$ -statement in α^{od} and the $\sqcup$ -statement in α are not merely equivalent: they are the *same* proposition, which is why every dual lemma in `Diagonal.lean` is proved by its primal with the single annotation $(\alpha := \alpha^{\text{od}})$.

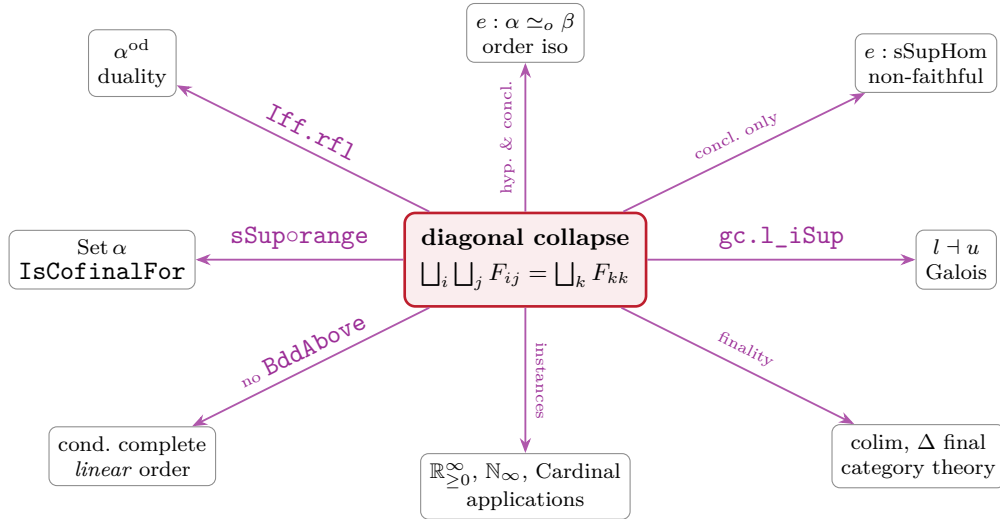


Figure 19: One theorem, eight contexts. The label on each arrow records how much of the statement survives the transport — the whole point of the bridge methodology is that the cheapest context is not always the one the statement was posed in.

4 The space of proofs of one lemma

`RequestProject/Variations.lean` proves each result of the development several times, moving along the axes named in the proof-writing guides. Those axes span a Boolean lattice, so the catalogue of variants is literally a set of vertices of a cube — and one can see at a glance which corners are occupied.

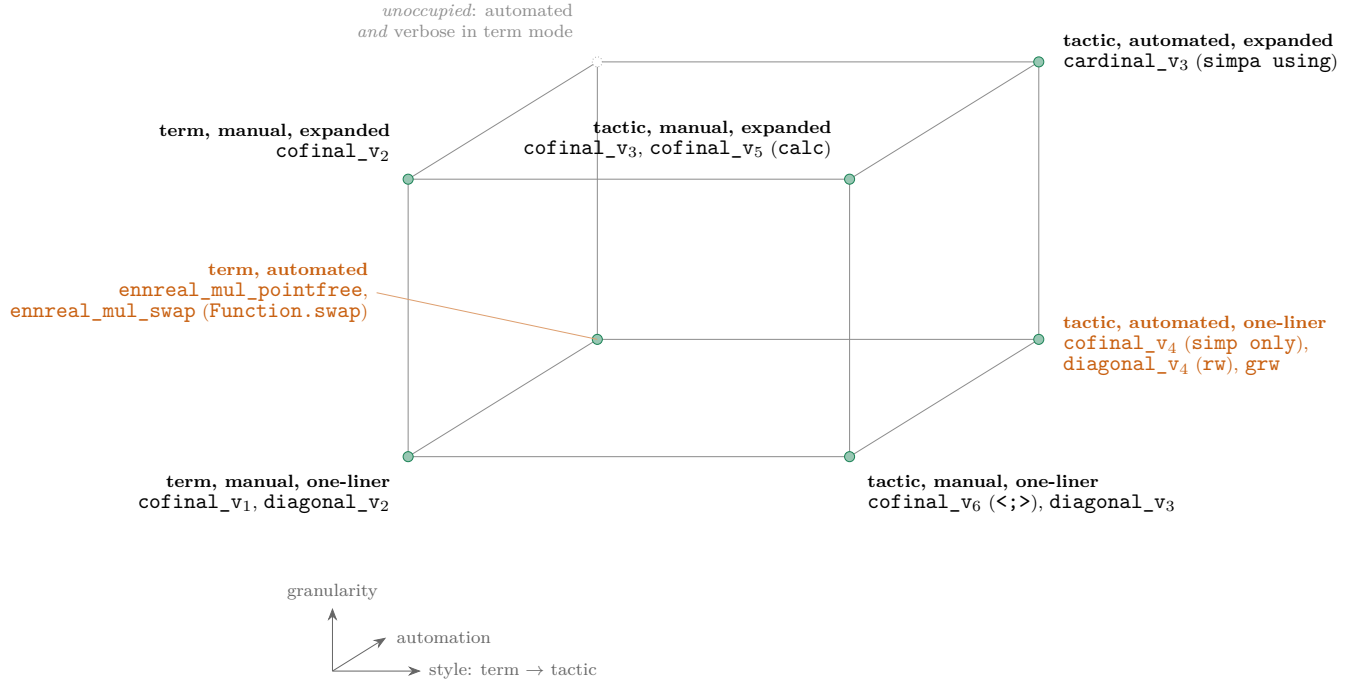


Figure 20: The design space of the mutual-cofinality lemma as the Boolean lattice B_3 . Green vertices are realised by compiled proofs in `Variations.lean`.

4.1 A fourth axis: duality

Duality is not a point of the cube but an involution acting on the whole of it: applying $(-)^{\text{od}}$ to any proof yields a proof of the $\sqcap$ -statement, and Lean accepts it with a single type ascription.

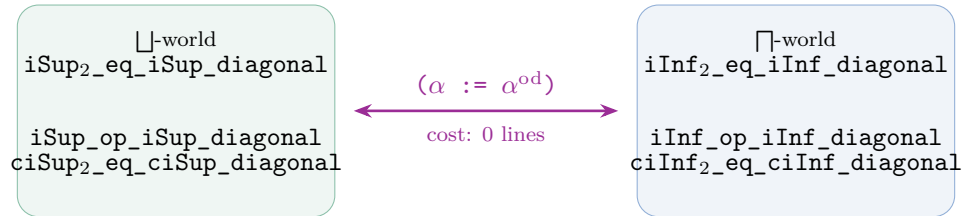


Figure 21: The dual half of `Diagonal.lean` is generated, not written.

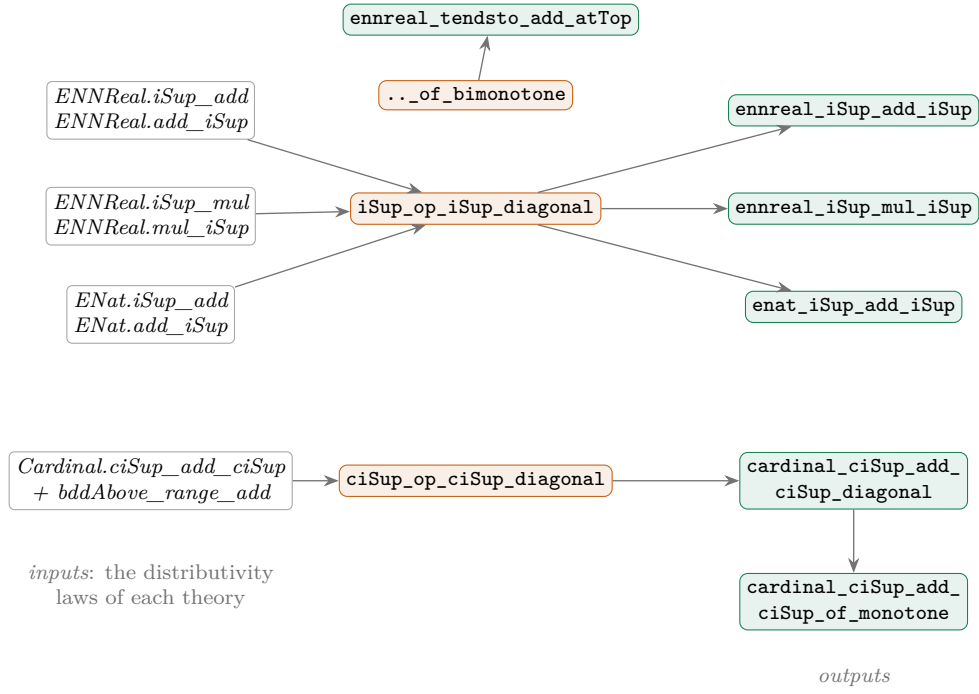


Figure 22: The operation-level packaging turns each application into “supply the two distributivity lemmas”. The multiplicative statement for $\mathbb{R}_{\geq 0}^{\infty}$ and the additive one for $\mathbb{N}_{\infty}$ cost two lines each, once the abstraction exists.

4.2 From the abstraction to the applications

4.3 Inventory

Statement	Axes exercised	Variants
mutual cofinality	term/tactic, forward/backward, <code>calc</code> , <code><;></code>	7
diagonal corollary	corollary, standalone, <code>rw</code> , duality	6
binary-operation form	<code>trans</code> chain, <code>simp_rw</code> , point-free	4
boundedness helpers	set-level vs. family-level	5
conditionally complete collapse	currying-first vs. <code>le_antisymm</code> -first	5
Cardinal payoff	<code>rw</code> , <code>simp</code> using, linear-order bridge	5
$\mathbb{R}_{\geq 0}^{\infty}$	operation-level, categorical, <code>sSupHom</code> transport	6
structural atoms	term/tactic, manual/automated	6